

CSE 4403: Algorithms  
Lecture 23

# Randomized Algorithms

Anika Farzana

Junior Lecturer

Department of Computer Science & Engineering

Islamic University of Technology, Dhaka, Bangladesh

Email: [anikafarzana@iut-dhaka.edu](mailto:anikafarzana@iut-dhaka.edu)

# What Is a Randomized Algorithm?

A randomized algorithm makes some of its decisions using a randomly generated number  $r$ , drawn from a range such as  $(1, \dots, R)$ , instead of always following one fixed rule.

## Random Choice

At each decision point, the algorithm generates  $r$  and lets its value steer the next step, rather than always taking the same branch.

## Recursive Use

In a recursive algorithm, a fresh random number can be generated at every level of recursion, so each call makes its own independent random choice.

The randomness changes the algorithm's behavior across runs, which is exactly what makes its analysis interesting.

# Three Families of Randomized Algorithms

Randomized algorithms are usually grouped by what the randomness is allowed to affect: the running time, the correctness of the output, or both.

1

## Monte Carlo Algorithm

Always finishes in a fixed amount of time, but the output is only correct with some probability.

2

## Las Vegas Algorithm

Always outputs the correct answer if it terminates, but its running time varies from run to run.

3

## Atlantic City Algorithm

Runs fast most of the time and is usually correct, blending traits of the other two.

# Origin of Monte Carlo: A Dice Example

Before turning to randomness as a tool, consider a simple question: what is the probability of getting an even number from a single throw of a fair die? When the sample space is small, the exact answer is easy to work out.

Sample space = {1, 2, 3, 4, 5, 6} → Favorable outcomes = {2, 4, 6}

$$P(\text{even}) = 3 / 6 = 0.5 \text{ (50\%)}$$

This works because we can enumerate the whole sample space by hand, but that will not stay true for long.

# Origin of Monte Carlo: Ulam's Solitaire Problem

Stanisław Ulam first thought about this class of problem while recovering from an illness, when he wondered: what is the probability of winning a game of solitaire from a randomly shuffled deck?

## Why Exact Counting Fails

A 52-card deck can be shuffled in  $52! \approx 10^{68}$  distinct ways, far too many to enumerate directly.

## Just How Large Is That?

Every one of Earth's  $\approx 10^{10}$  people, working every second since the Big Bang ( $\approx 10^{17}$  seconds), still would not approach  $10^{68}$ .

Rather than finding the exact number, Ulam realized it is far cheaper to simulate many games and count how many are won.

# From Idea to Algorithm: Monte Carlo Methods

The same simulate-and-count idea from the solitaire problem generalizes into a broad family of techniques, the Monte Carlo methods. Two examples make the idea concrete:

1

## Approximating $\pi$

Use repeated random sampling over a geometric region to estimate the value of  $\pi$ .

2

## Expected Number of Rounds

Use repeated random simulation to estimate how many rounds a probabilistic game is expected to take.

# Approximating $\pi$ : The Geometric Setup

Take a circle of radius  $r$  inscribed in a square of side length  $2r$ . The ratio of the circle's area to the square's area depends only on  $\pi$ , which gives a way to recover  $\pi$  from that ratio.

$$x = \pi r^2 / 4r^2$$

$$\pi = 4x$$

If we can estimate the area ratio  $x$  by another means, we can estimate  $\pi$  from it.

# Approximating $\pi$ with Monte Carlo Simulation

APPROXIMATE-PI(T):

$C = 0$

    for  $i = 1$  to  $T$ :

        generate random  $x, y$  in  $[-1, 1]$

        if  $x^2 + y^2 \leq 1$ :

$C = C + 1$

    return  $4 * (C / T)$

// A point  $(x, y)$  lies inside the unit circle exactly when  
// its distance from the center satisfies  $\sqrt{x^2 + y^2} \leq 1$ .  
// The fraction of points landing inside approximates the  
// area ratio  $\pi$  from the geometric setup.

# Approximating $\pi$ with Monte Carlo Simulation

APPROXIMATE-PI(T):

$C = 0$

for  $i = 1$  to  $T$ :

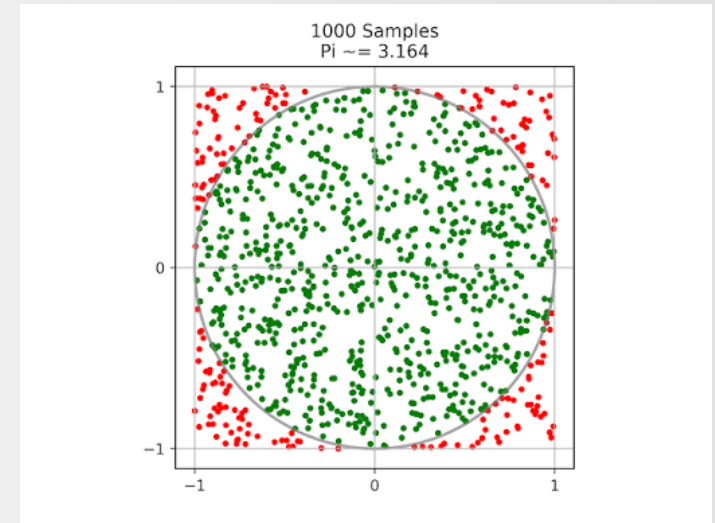
    generate random  $x, y$  in  $[-1, 1]$

    if  $x^2 + y^2 \leq 1$ :

$C = C + 1$

return  $4 * (C / T)$

// A point  $(x, y)$  lies inside the unit circle exactly when  
// its distance from the center satisfies  $\sqrt{x^2 + y^2} \leq 1$ .  
// The fraction of points landing inside approximates the  
// area ratio  $\pi$  from the geometric setup.



# Expected Number of Rounds: Problem Setup

Consider a game played round after round that ends as soon as the player loses two rounds in a row. Each round is won independently with probability  $p$ , regardless of prior outcomes. How many rounds do we expect the game to last?

## Game Rule

The game stops the first time the player records two consecutive losses.

## Goal

Find  $E$ , the expected total number of rounds played before the game ends.

We will solve this both with an exact recurrence and, separately, with a Monte Carlo simulation.

## Aside: What Is Expected Value?

The expected value of an uncertain outcome is the probability-weighted average of all its possible values.

$$EV = \sum P(X_i) \times X_i$$

$$EV(\text{Project A}) = 0.4 \times \$2,000 + 0.6 \times \$500 = \$1,100$$

$$EV(\text{Project B}) = 0.3 \times \$3,000 + 0.7 \times \$200 = \$1,040$$

Project A has the higher expected value, even though neither outcome is guaranteed.

## Expected Rounds: Setting Up the Recurrence

Conditioning on the outcome of the first two rounds gives a recurrence for  $E$ , which solves to a closed form.

$$E = (1+E) \cdot p + 2 \cdot (1-p)(1-p) + (2+E) \cdot (1-p) \cdot p$$

$$E = (2 - p) / (1 - p)^2$$

Where  $E$  = Expected number of rounds until two consecutive losses  
 $p$  = the winning probability in a single round

# Expected Rounds: Case Analysis

Conditioning on the first one or two rounds splits the game into three mutually exclusive cases:

1

**First round is a win (prob.  $p$ )**

The two-loss streak is broken immediately, so the rest of the game restarts fresh. Total rounds:  $1 + E$ .

2

**First round loses, second round also loses (prob.  $(1-p)^2$ )**

Two consecutive losses end the game right there. Total rounds: 2.

3

**First round loses, second round wins (prob.  $(1-p) \cdot p$ )**

The losing streak is broken, so the rest of the game restarts fresh. Total rounds:  $2 + E$ .

# Expected Rounds via Monte Carlo Simulation

```
EXPECTED-ROUNDS(p, N):  
    total = 0  
    for i = 1 to N:  
        nloss = 0  
        nround = 0  
        while nloss != 2:  
            nround = nround + 1  
            r = random value in [0, 1]  
            if r > p:  
                nloss = nloss + 1  
            else:  
                nloss = 0  
        total = total + nround  
    return total / N          // average number of rounds
```

# Pros and Cons of Monte Carlo

Trading exactness for repeated sampling has clear upsides, and equally clear costs.

## Advantages

An analytical solution can be hard to derive and may need deep domain knowledge that Monte Carlo does not require. It is usually faster to get a usable answer, and the design itself is quick to build.

## Drawbacks

Runtime can be very high when convergence is slow (e.g. a win probability near 1). The simulation gives no insight into why the answer holds, reuse for a new input needs a full re-run, and conclusions may not generalize.

# Las Vegas Algorithms

A Las Vegas algorithm is guaranteed to produce the correct result whenever it produces a result at all. It never returns a wrong answer, only no answer.

1

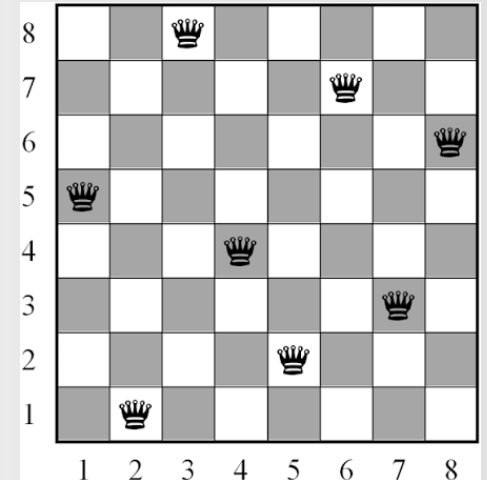
## 8 Queens Problem

Place queens one row at a time, choosing randomly among the valid positions in each row.

2

## Randomized Quick Sort

Pick the pivot for each partition step at random rather than always using a fixed position.



# 8 Queens: A Las Vegas Approach

**1****For each row  $k = 1$  to 8**

Consider the queen that must be placed in row  $k$ .

**2****Find valid positions in row  $k$** 

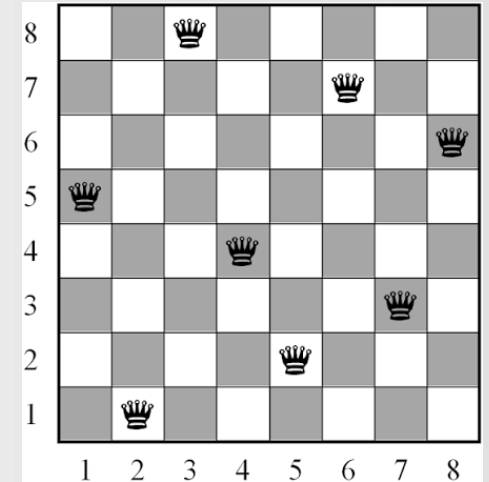
Determine every column in row  $k$  where a queen would not attack any queen already placed.

**3****Choose or fail**

If no valid position exists, report failure. If more than one is valid, pick one uniformly at random.

**4****Check completion**

If  $k = 8$ , a full valid placement has been found, so return it as the solution.



## 8 Queens: A Las Vegas Approach

1

**For each row  $k = 1$  to 8**

Consider the queen that must be placed in row  $k$ .

2

**Find valid positions in row  $k$**

Determine every column in row  $k$  where a queen would not attack any queen already placed.

3

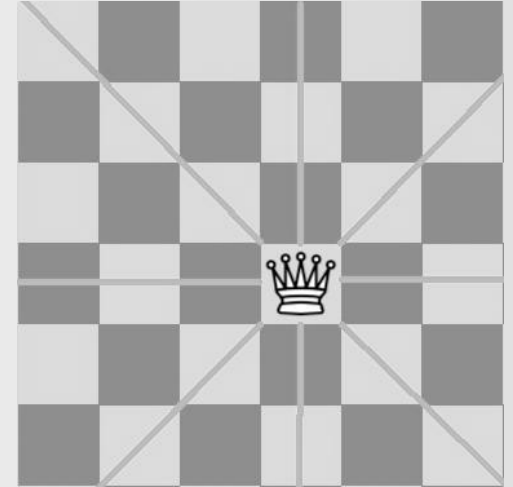
**Choose or fail**

If no valid position exists, report failure. If more than one is valid, pick one uniformly at random.

4

**Check completion**

If  $k = 8$ , a full valid placement has been found, so return it as the solution.



## Randomized Quick Sort: Main Routine

```
QUICKSORT(arr, low, high):  
    if low < high:  
        pi = PARTITION(arr, low, high)  
        QUICKSORT(arr, low, pi - 1)    // before pi  
        QUICKSORT(arr, pi + 1, high)   // after pi
```

The recursive structure is ordinary quicksort, since the randomness lives entirely inside partitioning.

# Quick Sort: The Partition Routine

```
PARTITION(arr, low, high):  
    pivot = arr[high]    // could be any chosen position  
    i = low  
    for j = low to high - 1:  
        if arr[j] < pivot:  
            swap arr[i] and arr[j]  
            i = i + 1  
    swap arr[i] and arr[high]  
    return i
```

# Randomizing the Pivot Choice

```
PARTITION-RANDOM(arr, low, high):  
    r = random value between low and high  
    swap arr[r] and arr[high]  
    return PARTITION(arr, low, high)
```

Randomizing the pivot removes the dependence on input order.  
No fixed input can reliably trigger the  $O(n^2)$  worst case, so the  
**expected running time becomes  $O(n \log n)$ .**